Université Gustave Eiffel

Laboratoire d'Informatique Gaspard Monge

Space Invader

Computer Vision

Auteurs : Badredin BELACHHEB & Janelle TOURNEUR & Jeremy SARRI & Lisa PIN

Encadrant : Florian VALADE

Champs-sur-Marne — Année universitaire 2025–2026

Table des matières

Computer Vision : Space Invaders **1**

1 Bases . . . 1
- 1.1 Neurone artificiel . . . 1
- 1.2 Perceptron . . . 2

2 Convolutional Neural Network . . . 4
- 2.1 Architecture du CNN . . . 5
- 2.2 Convolution . . . 5
- 2.3 Max-Pooling . . . 7
- 2.4 ReLU . . . 8

3 Collecte des données . . . 11

4 Première approche : CNN . . . 12
- 4.1 Paramètres . . . 12
- 4.2 Architecture du modèle CNN . . . 12
- 4.3 Principe d'apprentissage supervisé du modèle . . . 14
- 4.4 Boucle d'entraînement et validation . . . 14
- 4.5 Limites du modèle . . . 15

5 Deuxième approche : YOLO . . . 17
- 5.1 Entraînement du modèle . . . 17
- 5.2 Vérifications . . . 18

6 Troisième approche : MediaPipe . . . 20
- 6.1 Principe général . . . 20
- 6.2 Limites observées . . . 23

Computer Vision : Space Invaders

1 Bases

1.1 Neurone artificiel

Mathématiquement, un neurone est une fonction

$$f_\omega : \mathbb{R}^n \to \mathbb{R}, \qquad x \mapsto \sigma(Wx + b)$$

Le vecteur d'entrée $x = (x_1, \ldots, x_n)$ représente les caractéristiques du problème. Chaque entrée est associée à un poids w_i qui mesure l'importance de cette variable dans la décision du neurone.

La première étape consiste à calculer une combinaison linéaire des entrées :

$$z = Wx + b = \sum_{i=1}^{n} w_i x_i + b$$

Le terme b est appelé *biais* et permet de décaler la fonction de décision.

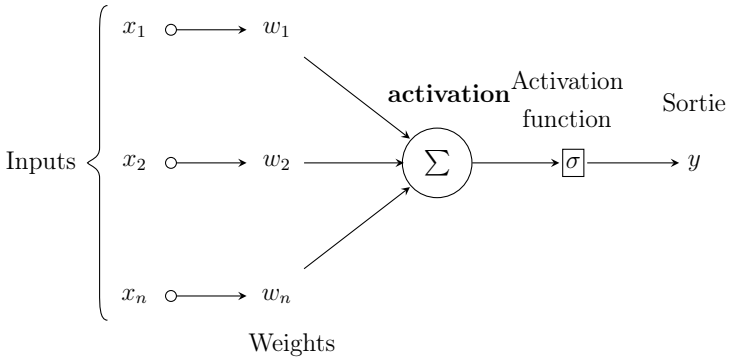
Ensuite, cette quantité z est transformée par une fonction d'activation σ. Cette fonction introduit une non-linéarité dans le modèle, ce qui permet aux réseaux de neurones d'approximer des relations complexes entre les variables.

Finalement, la sortie du neurone est donnée par

$$y = \sigma(z)$$

Ce mécanisme simple constitue l'unité de base des réseaux de neurones profonds, où plusieurs neurones sont empilés en couches afin d'apprendre des représentations de plus en plus abstraites des données.

Voici un graphique résumant son fonctionnement :



1.2 Perceptron

Un perceptron est un réseau de neurones composé de deux couches :

1. une couche cachée dont tous les neurones partagent la même fonction d'activation non linéaire, généralement une sigmoïde,
2. une couche de sortie linéaire.

Le modèle est caractérisé par plusieurs paramètres structurels :

— n : le nombre de variables d'entrée,

— k : le nombre de neurones dans la couche cachée,

— p : le nombre de neurones dans la couche de sortie.

Les paramètres du réseau sont regroupés dans deux matrices :

— $W \in \mathcal{M}_{k,n}$ qui contient les poids reliant l'entrée à la couche cachée,

— $O \in \mathcal{M}_{p,k}$ qui contient les poids reliant la couche cachée à la couche de sortie.

Pour un vecteur d'entrée $x \in \mathbb{R}^n$, le réseau applique la transformation suivante :

$$R(x) = O\, s(Wx)$$

où $s : \mathbb{R} \to \mathbb{R}$ est une fonction sigmoïde appliquée composante par composante.
Le calcul se déroule donc en deux étapes :

1. une transformation linéaire Wx suivie de l'application de la fonction d'activation s, produisant les activations de la couche cachée,
2. une combinaison linéaire de ces activations via la matrice O pour produire la sortie finale.

Les matrices W et O constituent les paramètres du réseau, appelés *poids*, qui seront appris à partir des données lors de la phase d'entraînement.

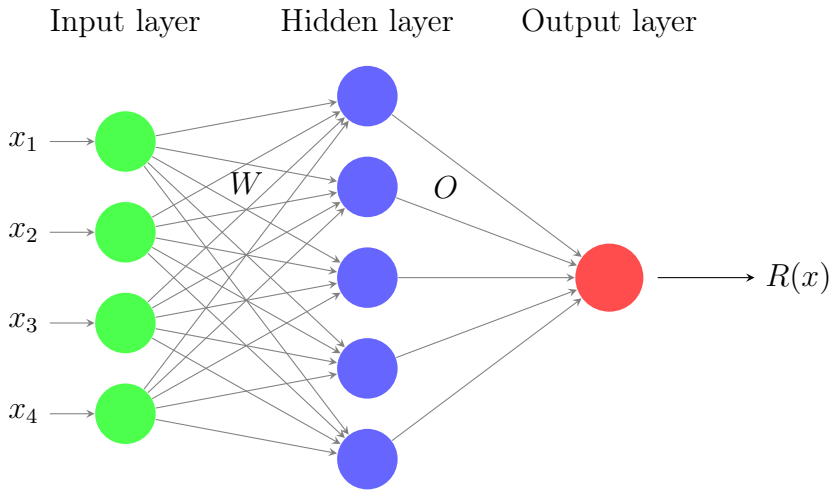


FIGURE 1 – Perceptron with $n = 4$, $k = 5$ and $p = 1$

2 Convolutional Neural Network

Un réseau neuronal convolutif s'inspire de la nature, car la connectivité entre les neurones artificiels ressemble à l'organisation du cortex visuel animal. L'un des principaux cas d'usage est la reconnaissance d'image. Les réseaux convolutifs apprennent plus rapidement et ont un meilleur taux d'erreur. Dans une moindre mesure, on les utilise aussi pour l'analyse vidéo. Ce type de réseau est aussi utilisé pour le traitement naturel du langage. Les modèles CNN sont très efficaces pour l'analyse sémantique, la modélisation de phrase, la classification ou la traduction.

Les CNN désignent une sous-catégorie de réseaux de neurones et sont à ce jour un des modèles de classification d'images réputés être les plus performant. Leur mode de fonctionnement est à première vue simple : l'utilisateur fournit en entrée une image sous la forme d'une matrice de pixels.

Celle-ci dispose de 3 dimensions :

— Deux dimensions pour une image en niveaux de gris.

— Une troisième dimension, de profondeur 3 pour représenter les couleurs fondamentales (Rouge, Vert, Bleu).

Contrairement à un modèle MLP (Multi Layers Perceptron) classique qui ne contient qu'une partie classification, l'architecture du Convolutional Neural Network dispose en amont d'une partie convolutive et comporte par conséquent deux parties bien distinctes :

— Une partie convolutive : Son objectif final est d'extraire des caractéristiques propres à chaque image en les compressant de façon à réduire leur taille initiale. En résumé, l'image fournie en entrée passe à travers une succession de filtres, créant par la même occasion de nouvelles images appelées cartes de convolutions. Enfin, les cartes de convolutions obtenues sont concaténées dans un vecteur de caractéristiques appelé code CNN.

— Une partie classification : Le code CNN obtenu en sortie de la partie convolutive est fourni en entrée dans une deuxième partie, constituée de couches entièrement connectées appelées perceptron multicouche (MLP pour Multi Layers Perceptron). Le rôle de cette partie est de combiner les caractéristiques du code CNN afin de classer l'image.

Contrairement à un modèle MLP classique, l'ajout en amont de la partie convolutive permet d'obtenir en sortie une « carte de caractéristiques » ou « code CNN » dont les dimensions sont plus petites que celles de l'image initiale ce qui va avoir l'avantage de diminuer grandement le nombre de paramètres à calculer dans le modèle.

2.1 Architecture du CNN

Généralement, l'architecture d'un Convolutional Neural Network peut être décrit ainsi, en deux couches que l'on ajoute successivement :

— Couche de convolution (CONV) : Le rôle de cette première couche est d'analyser les images fournies en entrée et de détecter la présence d'un ensemble de features. On obtient en sortie de cette couche un ensemble de features maps.

— Couche de Pooling (POOL) : La couche de Pooling est une opération généralement appliquée entre deux couches de convolution. Celle-ci reçoit en entrée les features maps formées en sortie de la couche de convolution et son rôle est de réduire la taille des images, tout en préservant leurs caractéristiques les plus essentielles. Parmi les plus utilisés, on retrouve le max-pooling mentionné précédemment ou encore l'average pooling dont l'opération consiste à conserver à chaque pas, la valeur moyenne de la fenêtre de filtre.

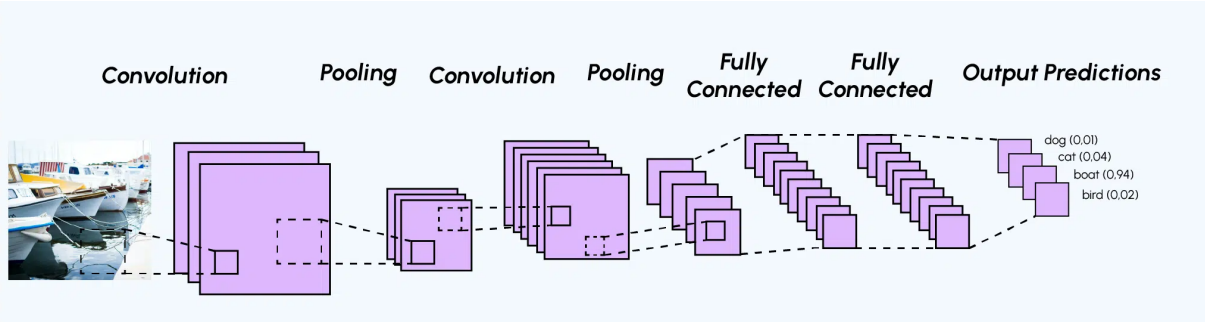


FIGURE 2 – Architecture CNN

2.2 Convolution

La convolution est une opération mathématique simple généralement utilisée pour le traitement et la reconnaissance d'images. Sur une image, son effet s'assimile à un filtrage dont voici le fonctionnement :

1. Dans un premier temps, on définit la taille de la fenêtre de filtre située en haut à gauche.
2. La fenêtre de filtre, représentant la feature, se déplace progressivement de la gauche vers la droite d'un certain nombre de cases défini au préalable (le pas) jusqu'à arriver au bout de l'image.
3. À chaque portion d'image rencontrée, un calcul de convolution s'effectue permettant d'obtenir en sortie une carte d'activation ou feature map qui indique où est localisées les features dans l'image : plus la feature map est élevée, plus la portion de l'image balayée ressemble à la feature.

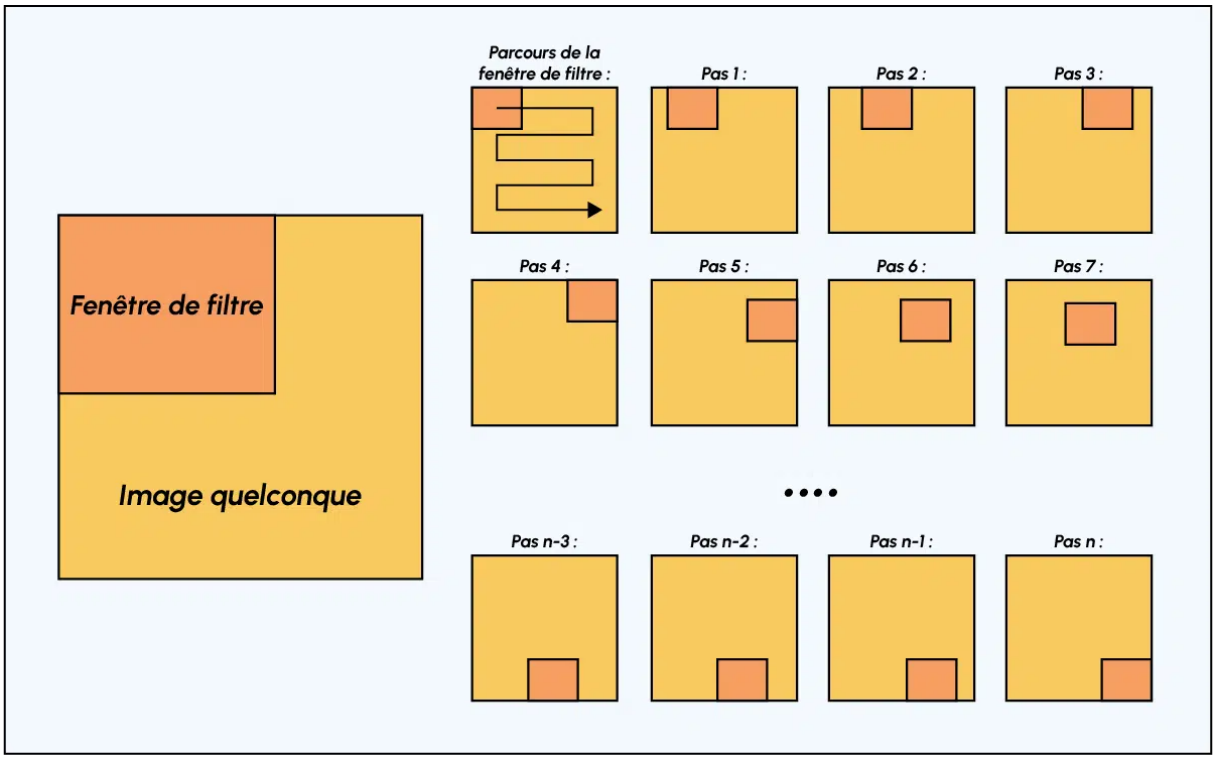


FIGURE 3 – Convolution

Exemple de convolution :

Lors de la partie convolutive d'un Convolutional Neural Network, l'image fournie en entrée passe à travers une succession de filtres de convolution. Par exemple, il existe des filtres de convolution fréquemment utilisés et permettant d'extraire des caractéristiques plus pertinentes que des pixels comme la détection des bords (filtre dérivateur) ou des formes géométriques. Le choix et l'application des filtres se fait automatiquement par le modèle.

Parmi les filtres les plus connus, on retrouve notamment le filtre moyenneur (calcule pour chaque pixel la moyenne du pixel avec ses 8 proches voisins) ou encore le filtre gaussien permettant de réduire le bruit d'une image fournie en entrée :

Voici un exemple des effets de ces deux différents filtres sur une image comportant un bruit important (on peut penser à une photographie prise avec une faible luminosité par exemple). Toutefois, un des inconvénients de la réduction du bruit est qu'elle s'accompagne généralement d'une réduction de la netteté :

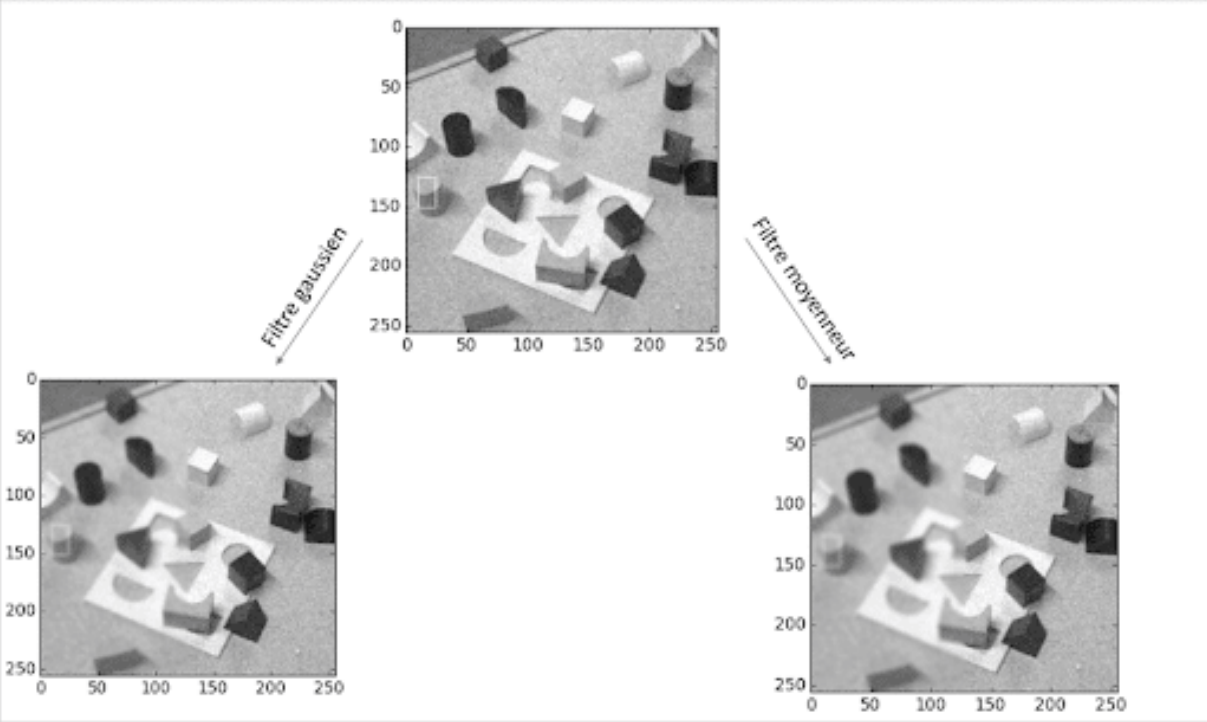


FIGURE 4 – Résultats de filtres

Outre sa fonction de filtrage, l'intérêt de la partie convolutive d'un CNN est qu'elle permet d'extraire des caractéristiques propres à chaque image en les compressant de façon à réduire leur taille initiale, via des méthodes de sous-échantillonnage tel que le Max-Pooling.

2.3 Max-Pooling

Le Max-Pooling est un processus de discrétisation basé sur des échantillons. Son objectif est de sous-échantillonner une représentation d'entrée (image, matrice de sortie de couche cachée, etc.) en réduisant sa dimension. De plus, son intérêt est qu'il réduit le coût de calcul en réduisant le nombre de paramètres à apprendre et fournit une invariance par petites translations [1].

1. Si une petite translation ne modifie pas le maximum de la région balayée, le maximum de chaque région restera le même et donc la nouvelle matrice créée restera identique

Exemple :

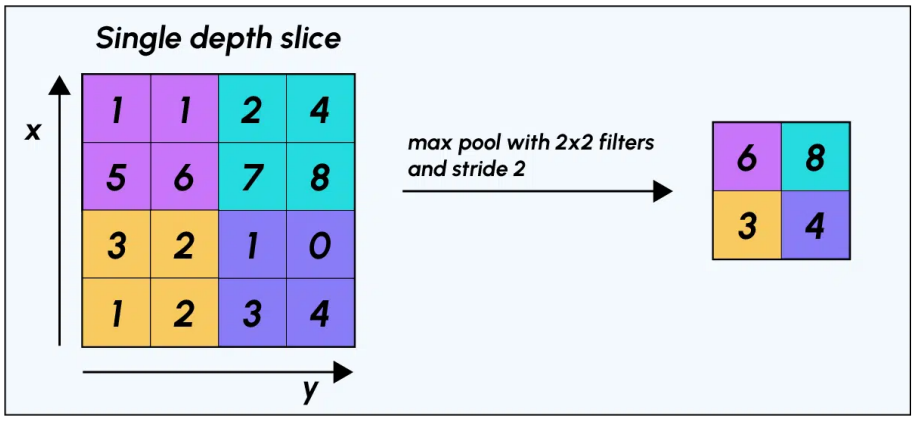


FIGURE 5 – Max-Pooling

La fenêtre de filtre se déplace de deux pixels vers la droite (stride/pas = 2) et récupère à chaque pas l'argmax correspondant à la valeur la plus grande parmi les 4 valeurs de pixels.

2.4 ReLU

Les fonctions d'activation jouent un rôle fondamental dans les réseaux de neurones. Elles permettent d'introduire de la non-linéarité dans le modèle, ce qui est indispensable pour capturer des relations complexes dans les données. En effet, sans fonction d'activation, un réseau composé uniquement de couches linéaires serait équivalent à une simple transformation linéaire, limitant fortement sa capacité de représentation.

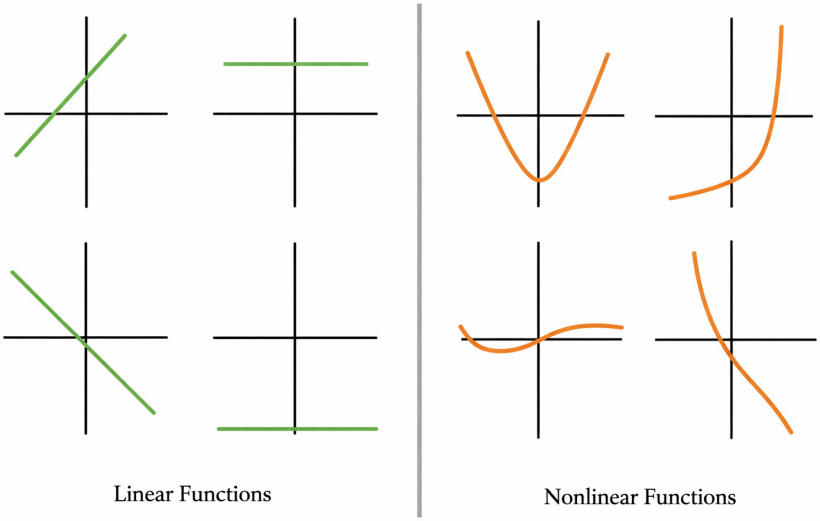


FIGURE 6 – Fonctions linéaires et non linéaires

Parmi les fonctions d'activation les plus utilisées, la fonction ReLU (*Rectified Linear Unit*) est devenue un standard dans les architectures de réseaux de neurones profonds, notamment dans les réseaux convolutionnels.

La fonction ReLU est définie mathématiquement par :

$$f(x) = \max(0, x)$$

Autrement dit, pour une entrée x :

— si $x > 0$, alors $f(x) = x$,

— si $x \leq 0$, alors $f(x) = 0$.

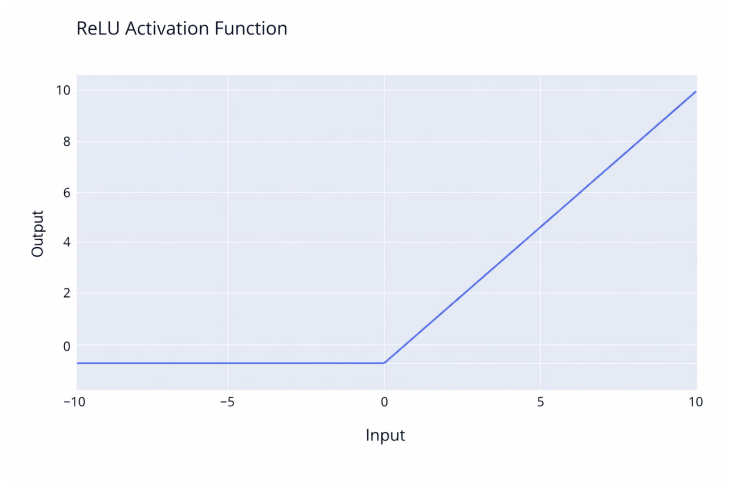


FIGURE 7 – Fonction d'activation ReLU

Cette fonction présente plusieurs avantages majeurs. Tout d'abord, elle introduit une non-linéarité simple qui permet au réseau d'apprendre des représentations complexes. Contrairement aux fonctions sigmoïde ou tangente hyperbolique, la ReLU ne sature pas pour les grandes valeurs positives, ce qui limite le problème de disparition du gradient lors de la rétropropagation.

En effet, dans les réseaux profonds, les gradients peuvent devenir très faibles lorsqu'ils traversent de nombreuses couches, ce qui ralentit voire empêche l'apprentissage. La ReLU atténue ce phénomène en maintenant un gradient constant égal à 1 pour les valeurs positives.

De plus, la ReLU produit des activations nulles pour les entrées négatives, ce qui conduit à une activation parcimonieuse du réseau. Cela signifie qu'à chaque instant, seule une partie des neurones est active, ce qui améliore l'efficacité computationnelle et réduit le coût de calcul.

Enfin, la simplicité de la fonction ReLU permet une implémentation rapide et efficace, ce qui en fait un choix privilégié dans la majorité des architectures modernes.

Dans le cadre de notre modèle CNN, la fonction ReLU est utilisée après chaque couche convolutionnelle ainsi que dans la partie de classification. Elle permet ainsi d'introduire de la non-linéarité entre les différentes transformations et permet au CNN d'apprendre des motifs non linéaires dans les images.

3 Collecte des données

Le script `collect_dataset.py` permet de créer notre dataset de photos qui servira à l'entraînement du modèle. Il active la webcam et enregistre l'image entière en la classant dans des dossiers selon le geste effectué et selon un split *train/validation*. L'utilisateur contrôle la collecte au clavier :

- Appuyer sur une touche entre `0` et `5` pour sélectionner une classe (`NONE`, `LEFT`, `RIGHT`, `FIRE`, `ENTER`, `QUIT`)
- Appuyer sur `c` pour activer ou désactiver la capture (5 images par seconde)
- Appuyer sur `v` pour basculer entre le mode *train* et le mode *validation*
- Appuyer sur `q` pour quitter le programme

Comme le CNN sera entraîné sur toute l'image, nous avons choisi des gestes très distincts afin de limiter les confusions entre les classes :

- `LEFT` : paume de la main gauche ouverte, avec les doigts collés ;
- `RIGHT` : paume de la main droite ouverte, avec les doigts écartés ;
- `FIRE` : poing de la main droite fermé ;
- `ENTER` : deux paumes ouvertes ;
- `QUIT` : croix avec les bras.

Nous avons ajouté une classe `NONE` où il n'y a pas de gestes à reconnaître, pour éviter que le CNN classe des images même lorsque l'utilisateur ne fait rien.

4 Première approche : CNN

Dans un premier temps, nous avons développé une approche basée sur un réseau de neurones convolutionnel (CNN) afin de classifier directement les images issues de la caméra en commandes de jeu.

Le modèle prend en entrée des images RGB redimensionnées en 224×224 et applique successivement plusieurs couches convolutionnelles, suivies de fonctions d'activation ReLU et d'opérations de max-pooling. Ces couches permettent d'extraire progressivement des caractéristiques visuelles pertinentes (formes, contours, textures).

L'architecture utilisée est composée de :

trois blocs convolution–ReLU–max-pooling, suivis d'une phase de classification fully connected, aboutissant à une sortie de dimension 6 correspondant aux actions possibles :

$$\{NONE, LEFT, RIGHT, FIRE, ENTER, QUIT\}$$

L'apprentissage est réalisé à partir d'un dataset d'images collectées via la webcam, avec des techniques d'augmentation de données (flip horizontal, normalisation) afin d'améliorer la robustesse du modèle.

Cette approche constitue une première validation du principe de reconnaissance de gestes à partir d'images brutes.

4.1 Paramètres

Le modèle fixe les paramètres suivants :

- *IMAGE_SIZE = 224* : toutes les images sont redimensionnées en 224×224.
- *BATCH_SIZE = 32* :32 images par lot.
- *NUM_EPOCHS = 10* : 10 passages complets sur les données.
- *LEARNING_RATE* = 10^{-3} : taux d'apprentissage.
- *CLASSES = ["NONE", "LEFT", "RIGHT", "FIRE", "ENTER", "QUIT"]* représentant une classification supervisée à 6 classes.

4.2 Architecture du modèle CNN

C'est la classe *GestureCNN* qui définit le réseau d'apprentissage.

Extraction des caractériques : self.features

Le réseau applique successivement 3 blocs convolutionnels :

Bloc 1	Bloc 2	Bloc 3
1. Convolution $3 \to 16$	1. Convolution $16 \to 32$	1. Convolution $32 \to 64$
2. ReLU	2. ReLU	2. ReLU
3. Max-pooling	3. Max-pooling	3. Max-pooling

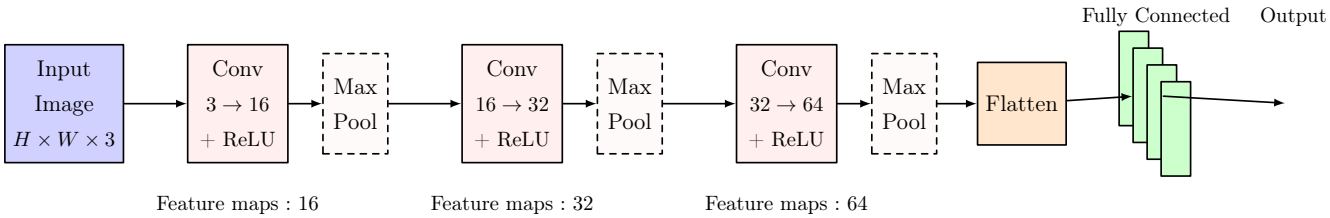


FIGURE 8 – Architecture du réseau de neurones convolutif utilisé : trois blocs convolutionnels, suivis d'une phase de *flatten*, d'une couche entièrement connectée, puis d'une couche de sortie.

Effet sur la taille

L'entrée est une image RGB de taille :

$$3 \times 224 \times 224.$$

Comme les convolutions ont `padding=1` et `kernel_size=3`, elles gardent la taille spatiale.

En revanche, chaque `MaxPool2d(2)` divise la hauteur et la largeur par 2 :

— après 1er pooling : $224 \to 112$,

— après 2e pooling : $112 \to 56$,

— après 3e pooling : $56 \to 28$.

Donc à la sortie de la partie convolutionnelle, on obtient une image de :

$$64 \times 28 \times 28.$$

Partie classification : `self.classifier`

Ensuite :

— `Flatten()` transforme le tenseur en vecteur,

— `Linear(64*28*28, 128)` crée une couche dense de 128 neurones,

— `ReLU()`,

— `Linear(128, num_classes)` produit les scores finaux pour les 6 classes.

Remarque : Le modèle ne prédit pas directement des probabilités, mais les logits [2].

4.3 Principe d'apprentissage supervisé du modèle

Le modèle est entraîné dans un cadre d'apprentissage supervisé. Les images du dataset sont organisées en sous-dossiers correspondant aux différentes classes de commandes du jeu (`NONE`, `LEFT`, `RIGHT`, `FIRE`, `ENTER`, `QUIT`). Lors du chargement des données, l'outil `ImageFolder` associe automatiquement à chaque image une étiquette correspondant au nom de son dossier.

Pour chaque image d'entrée, le réseau de neurones convolutionnel produit un vecteur de dimension 6, appelé *logits*, contenant un score pour chacune des classes possibles. Ces scores ne sont pas directement des probabilités, mais des valeurs relatives permettant de comparer les différentes classes.

Afin d'évaluer la qualité des prédictions, une fonction de perte de type entropie croisée (`CrossEntropyLoss` dans l'algorithme) est utilisée. Cette fonction compare les scores prédits par le modèle avec la véritable étiquette de l'image, et mesure l'écart entre la prédiction et la réalité.

L'apprentissage du modèle repose ensuite sur la rétropropagation du gradient. Plus précisément, le gradient de la fonction de perte est calculé par rapport aux paramètres du réseau, puis utilisé pour ajuster les poids via l'optimiseur Adam. Ce processus permet au modèle de renforcer progressivement les motifs visuels caractéristiques de chaque geste et d'améliorer ses performances au fil des itérations.

Enfin, lors de la phase d'inférence, la classe prédite correspond à celle associée au score maximal parmi les logits. Autrement dit, le modèle choisit la commande dont le score est le plus élevé :

$$\hat{y} = \arg\max_i f(x)_i$$

où $f(x)$ désigne la sortie du réseau pour une image x.

4.4 Boucle d'entraînement et validation

L'entraînement du modèle est réalisé sur plusieurs époques (`NUM_EPOCHS = 10`), chacune correspondant à un passage complet sur l'ensemble des données d'apprentissage. Les images sont chargées par mini-lots (*batches*) à l'aide de la classe `DataLoader`, ce qui permet d'optimiser le calcul et de stabiliser l'apprentissage.

À chaque itération, un lot d'images est transmis au modèle afin de produire des prédictions. La fonction de perte est ensuite calculée en comparant ces prédictions aux étiquettes réelles. Les gradients sont alors calculés par rétropropagation, puis les paramètres du modèle sont mis à jour à l'aide de l'optimiseur Adam.

2. Ce sont des scores que l'on converti ensuite en probabilités afin de faire la prédiction.

Au cours de l'entraînement, deux métriques principales sont suivies :

— la loss (erreur moyenne),

— l'accuracy (taux de bonnes prédictions).

Ces métriques sont calculées à la fois sur l'ensemble d'apprentissage (*train*) et sur un ensemble de validation (*val*). Ce dernier permet d'évaluer la capacité de généralisation du modèle sur des données non vues pendant l'apprentissage.

À la fin de chaque époque, une phase d'évaluation est effectuée sans calcul de gradient (`model.eval()` et `torch.no_grad()`). Les performances du modèle sur l'ensemble de validation sont alors mesurées.

Afin de conserver le meilleur modèle, une stratégie de sauvegarde est mise en place : si la précision sur l'ensemble de validation (`val_acc`) est supérieure à la meilleure valeur observée jusqu'à présent, les poids du modèle sont sauvegardés dans un fichier (`gesture_model.pth`).

Ainsi, le modèle final retenu correspond à celui qui offre les meilleures performances en validation, et non nécessairement à celui de la dernière époque.

4.5 Limites du modèle

La première approche basée sur un réseau de neurones convolutionnel constitue une preuve de concept efficace pour la reconnaissance de gestes à partir d'images issues de la webcam. Toutefois, une analyse plus approfondie met en évidence plusieurs limitations structurelles et méthodologiques.

Dépendance à l'image entière

Le modèle CNN est entraîné sur des images complètes sans mécanisme explicite de localisation de la main. Par conséquent, il apprend non seulement les caractéristiques du geste, mais également des éléments parasites tels que l'arrière-plan, la position du corps ou les conditions d'éclairage. Cette dépendance au contexte global peut nuire à la capacité de généralisation du modèle lorsque l'environnement change (fond différent, luminosité variable, position de la caméra, etc.).

Absence de focalisation sur la zone d'intérêt

Contrairement à des approches basées sur la détection ou la segmentation, le modèle ne dispose d'aucun mécanisme permettant de se concentrer explicitement sur la main. L'extraction des caractéristiques est donc réalisée sur l'ensemble de l'image, ce qui dilue l'information pertinente et peut dégrader les performances lorsque la main occupe une faible portion de l'image.

Limites des augmentations de données

L'augmentation de données joue un rôle essentiel dans la robustesse des modèles. Cependant, dans notre implémentation, l'utilisation d'un retournement horizontal aléatoire peut introduire une incohérence sémantique pour les classes directionnelles telles que `LEFT` et `RIGHT`. Une image représentant un geste vers la gauche peut être transformée en un geste visuellement similaire à celui vers la droite, sans modification de son étiquette, ce qui peut perturber l'apprentissage.

Capacité limitée du modèle

L'architecture utilisée reste relativement simple (trois blocs convolutionnels suivis d'une couche dense). Bien que suffisante pour une première approche, cette capacité limitée peut restreindre l'extraction de caractéristiques complexes, notamment en présence de variations importantes de posture, d'échelle ou d'orientation des gestes.

Sensibilité aux conditions d'acquisition

Le modèle est sensible aux variations de conditions d'acquisition (éclairage, qualité de la caméra, bruit, arrière-plan). En l'absence de normalisation avancée ou de mécanisme de robustesse spécifique, ses performances peuvent fortement dépendre du contexte dans lequel les données ont été collectées.

Ouverture

Face à ces limitations, il apparaît nécessaire d'adopter une approche plus robuste et mieux structurée. Dans ce cadre, nous introduisons une seconde méthode basée sur le modèle YOLOv8.

YOLO (*You Only Look Once*) est une architecture initialement conçue pour la détection d'objets en temps réel, reposant sur un réseau unique capable de prédire simultanément des localisations et des classes. Dans notre projet, nous utilisons la variante `YOLOv8n-cls`, qui adapte cette architecture à une tâche de classification d'images.

L'intérêt principal de cette approche réside dans l'utilisation d'un modèle préentraîné sur de larges bases de données, ainsi que dans un pipeline d'entraînement plus avancé intégrant des stratégies d'augmentation de données mieux contrôlées, un suivi expérimental plus riche et des mécanismes de régularisation tels que l'*early stopping*.

Ainsi, le passage à YOLOv8 permet d'améliorer la robustesse globale du système et la qualité de l'apprentissage, tout en conservant une approche de classification. Il constitue une évolution naturelle de notre première méthode, en offrant un cadre plus performant et plus adapté à des conditions d'utilisation variées.

5 Deuxième approche : YOLO

Dans un premier essai, nous avons choisi d'utiliser le modèle YOLO (You Only Look Once), plus particulièrement la version YOLOv8 qui est spécifique à la classification d'images/la reconnaissance de gestes.
L'avantage de ce modèle est qu'il traite l'image dans son intégralité et non en plusieurs étapes. Il fonctionne par grille i.e. l'image est découpée selon un quadrillage où chaque cellule prédit si un objet s'y trouve ou non. Sur chaque cellule, le modèle YOLO va prédire, simultanément, les bounding boxes, un score de confiance ainsi que la probabilité des classes.
Le modèle YOLO à l'avantage d'être rapide quant à la détection d'image, ce qui est primordial pour jouer au jeu Space Invaders, il est cependant moins précis sur la localisation exacte des contours. Il permet également une détection multiple, i.e. la détection dans notre cas des deux mains pour les commandes `ENTER` ou `QUIT` par exemple.

5.1 Entraînement du modèle

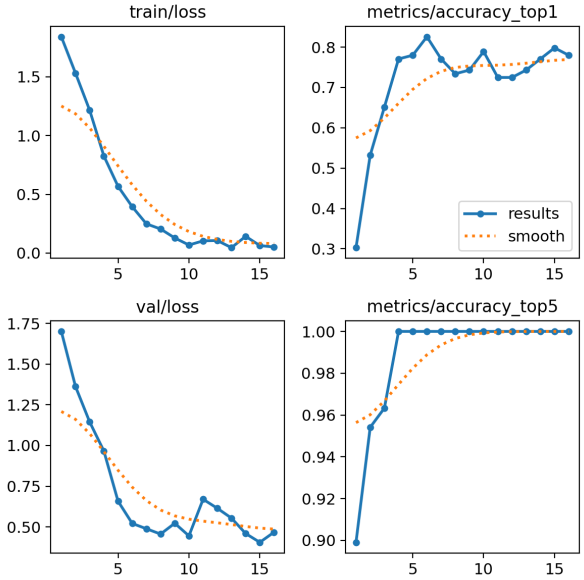
Le code d'entraînement du modèle est le code nommé "`train_model_yolo`". Ce code s'entraîne sur des data, que nous avons nous même recueillies grâce au code `collect_dataset`. Ce code nous a permis de créer une banque de photos sur laquelle le modèle va pouvoir s'entraîner.
Il comptabilise dans un premier temps le nombres photos prises pour entraîner ("train") et valider ("val") le modèle.

```
Dossier validation : val/

Classe          Train      Val
------------------------------
ENTER              26        8
FIRE               18       21
LEFT               35       17
NONE               36       15
QUIT               25       24
RIGHT              51       24
------------------------------
TOTAL             191      109

Dataset YOLO deja prepare dans model_output\yolo_dataset

Chargement du modele YOLOv8n-cls (nano, ~6MB)...
Entrainement (50 epochs max, early stopping patience=10)...
```

Notre dataset considéré n'est pas parfaitement équilibré (64% de données d'entraînement et 36% de données de validation, loin du standard 80% 20%) mais le modèle peut quand même s'entraîner.
Le modèle analyse des millier de fois les photos du sous-dossier "train" afin de s'entraîner à reconnaître les gestes (position des mains) et ensuite il considère les photos du sous-dossier "val" qu'il ne connaît pas pour vérifier s'il a su détecter correctement les gestes.

5.2 Vérifications

Nous pouvons voir si le modèle est bien entraîné grâce à des courbes d'apprentissages et des comparaisons de photos.



i) Le graphique *train-loss* (: erreur du modèle) i.e. la courbe d'erreur sur les images d'entraînement : elle décroît rapidement et fini près de 0.1, ce qui signifie que le modèle à bien mémorisé les gestes.

ii) Le graphique *val-loss* qui correspond à la courbe d'erreurs sur les images de validation décroît également, jusqu'à 0.5 puis devient stable, on voit donc que le modèle ne fait pas de sur-apprentissage.

iii) Le graphique *metrics-accuracy_ top1* (accuracy = le pourcentage de bonne réponse) atteint les 80% de bonnes réponses.

iv) Le graphique *metrics-accuracy_ top5* n'est pas super intéressante puisqu'elle dit qu'on se trouve dans le top 5 des bonnes réponses mais il y a seulement 6 possibilités (`LEFT, NONE, QUIT,...`) donc c'est peu surprenant..

Notre modèle YOLO, pendant sa phase d'entraîenemt, à enregistré des images qui nous permettent de voir comment il apprend.

L'image de gauche correspond aux **vraies** commandes, il n'y a donc aucune erreur sur les photos, elles correspondent à "l'objectif" du modèle. L'image de droite correspond aux résultats de notre modèle. On constate quelques erreurs, mais avec les courbes plus hautes on peut tout de même conclure que le modèle à fournit un travail relativement correct.
Enfin, notre code `train_model_yolo`, pour s'entraîner au mieux, s'auto crée des contraintes :

Ce qui lui permet d'anticiper d'éventuels changements de conditions (luminosité, cadrage ..) de sorte à être plus robuste.
Il aurait fallut un autre code afin de pouvoir jouer au jeu une fois le modèle entraîné mais nous avons décidé de changer d'approche.

6 Troisième approche : MediaPipe

Dans la deuxième approche, nous avons utilisé un modèle basé sur *YOLO* afin de détecter et classifier les gestes à partir de l'image brute. Bien que cette méthode soit performante en détection, elle présente certaines limites dans notre contexte : sensibilité aux conditions d'éclairage, dépendance à la qualité des données d'entraînement, et manque de stabilité dans la reconnaissance fine des gestes. Afin de pallier ces limitations, nous proposons une approche complémentaire basée sur l'analyse géométrique de la main via la librairie *MediaPipe*. Contrairement à *YOLO*, cette méthode exploite directement les points caractéristiques de la main, ce qui permet de corriger certaines erreurs du modèle précédent, notamment en termes de robustesse, d'invariance à l'échelle et de précision directionnelle. L'objectif de cette approche est donc de raffiner et stabiliser la détection des commandes, en s'affranchissant des contraintes liées à l'apprentissage supervisé et en exploitant une représentation plus structurée du geste. Dans cette troisième approche, nous utilisons la bibliothèque *MediaPipe Hands* développée par Google afin d'extraire en temps réel les 21 points caractéristiques (landmarks) de la main. Ces points sont fournis en coordonnées normalisées (x, y, z) et permettent de reconstruire la géométrie de la main indépendamment de sa position dans l'image.

6.1 Principe général

À partir de ces landmarks, nous définissons un algorithme de reconnaissance de gestes basé sur :

- la direction du doigt index,
- la configuration relative des doigts,
- des distances normalisées pour garantir l'invariance à l'échelle.

Contrairement à une approche basée sur l'image brute (CNN), cette méthode repose uniquement sur la géométrie de la main, ce qui la rend :

- plus robuste aux variations d'éclairage,
- indépendante de la position globale de la main,
- plus légère en calcul.

Normalisation et invariance d'échelle

Afin de rendre les seuils indépendants de la taille de la main ou de la distance à la caméra, nous introduisons un facteur d'échelle :

$$\text{scale} = \|\text{thumb_ip} - \text{thumb_tip}\|$$

Toutes les distances et seuils sont ensuite exprimés relativement à cette quantité.

Cette normalisation permet d'obtenir des critères invariants par homothétie, garantissant que les gestes sont reconnus de manière cohérente quelle que soit la position de l'utilisateur.

Définition des gestes

Les commandes sont déterminées à partir de la position relative de l'index :

— **Repos (aucune commande)** :

$$\|\text{index_tip} - \text{thumb_tip}\| < \text{scale}$$

Cette pose correspond à un geste naturel de type "OK" permettant de désactiver l'envoi de commandes.

— **Bas (ENTER)** :

$$(\text{index_tip}_y - \text{index_mcp}_y) > \alpha \cdot \text{scale}$$

— **Haut (FIRE)** :

$$(\text{index_tip}_y - \text{index_mcp}_y) < -\alpha \cdot \text{scale}$$

— **Gauche (LEFT)** :

$$(\text{index_tip}_x - \text{index_mcp}_x) > \alpha \cdot \text{scale}$$

— **Droite (RIGHT)** :

$$(\text{index_tip}_x - \text{index_mcp}_x) < -\alpha \cdot \text{scale}$$

où $\alpha = 2.6$ est un seuil empirique permettant de discriminer les directions.

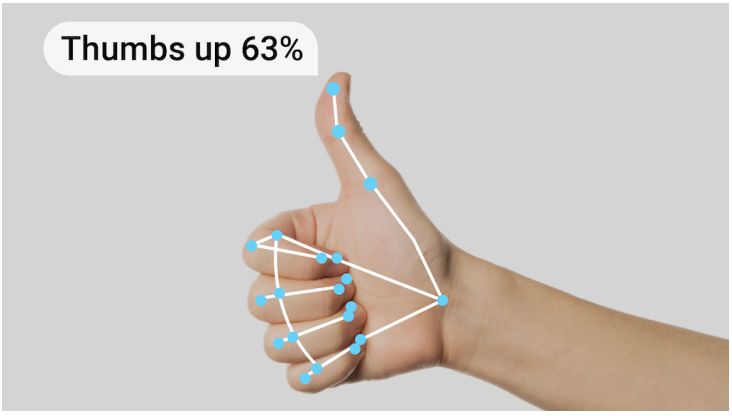


FIGURE 9 – Analyse des gestes avec *MediaPipe*

Extension : geste de sortie

Un geste supplémentaire est introduit pour quitter l'application :

— **Quitter ("a")** : lorsque les doigts majeur et annulaire sont levés simultanément avec une certaine séparation, ce qui correspond à un geste de type *salut vulcain*.

Ce critère repose sur :

— une grande distance entre les extrémités des doigts,

— une orientation verticale vers le haut.

Implémentation

L'algorithme est implémenté via la fonction suivante :

```
def input_to_command(lmdhands):
    ldms=lmdhands[0].landmark

    if len(ldms)<21: return None
    index_tip=ldms[8]
    index_mcp=ldms[5]
    thumb_tip=ldms[4]
    thumb_ip=ldms[3]
    thumb_mcp=ldms[2]
    middle_finger_tip=ldms[12]
    middle_finger_mcp=ldms[9]
    ring_finger_tip=ldms[16]
    ring_finger_mcp=ldms[13]

    scale=dist(thumb_ip,thumb_tip)
    alpha=2.6

    if dist(thumb_tip,index_tip) < scale:
        return None

    elif (dist(middle_finger_tip,ring_finger_tip)>(alpha/1.8)*scale) and ((
    middle_finger_tip.y-middle_finger_mcp.y)<-alpha*scale*1.2) and ((
    ring_finger_tip.y-ring_finger_mcp.y)<-alpha*scale*1.2):
        return "a"

    elif (index_tip.y-index_mcp.y)>alpha*scale:
        return "ENTER"

    elif (index_tip.y-index_mcp.y)<-alpha*scale:
        return "FIRE"

    elif ((index_tip.x-index_mcp.x)>alpha*scale):
        return "LEFT"

```

```
    elif ((index_tip.x-index_mcp.x)<-alpha*scale) or ((thumb_tip.x-thumb_mcp.x)<-alpha*0.5*scale):
        return "RIGHT"

    return None
```

Listing 1 – Détection des classes

6.2 Limites observées

Malgré de bonnes performances en détection et classification des gestes, plusieurs limitations ont été identifiées :

— **Problème de transmission** : les commandes ne sont pas toujours correctement reçues par le jeu.

— **Fréquence d'envoi trop élevée** : les instructions sont envoyées à chaque frame, ce qui peut saturer la communication WebSocket.

— **Absence de filtrage temporel** : les petites fluctuations des landmarks peuvent générer des commandes instables.

Améliorations possibles

Plusieurs pistes d'amélioration peuvent être envisagées :

— **Filtrage temporel** : validation d'un geste uniquement s'il est détecté sur plusieurs frames consécutives.

— **Réduction du débit** : limiter la fréquence d'envoi des commandes.

— **Utilisation d'angles** : remplacer les seuils cartésiens par une analyse directionnelle du vecteur de l'index.

— **Prise en compte de la main (gauche/droite)** : adapter les directions selon la main détectée.

Conclusion

Cette approche basée sur *MediaPipe* constitue une solution simple, rapide et interprétable pour le contrôle gestuel. Elle présente l'avantage d'être indépendante des données d'entraînement, contrairement aux méthodes basées sur les réseaux de neurones.

Cependant, sa robustesse dépend fortement du choix des seuils et de la stabilité temporelle, ce qui nécessite des améliorations pour une utilisation en conditions réelles.